

CST-407 Milestone 2 Implement JWT and REST API

Alex M. Frear

College of Science, Engineering, and Technology, Grand Canyon University

Course Number: CST-407

Professor: Dr. Melody White

08/26/2025

Link to Screencast: [CST407 - Milestone 2 - Implement JWT and REST API](#)

Table of Contents

1. Overview.....	4
2. Environment & Setup	5
2.1 Database configuration (datasource + JPA)	5
2.1.1 Database (MySQL)	5
2.1.2 JWT (stateless API security).....	5
2.1.3 Quick validation checklist.....	6
2.2 How to run (port 8080, MySQL schema)	6
2.2.1 Prerequisites	6
2.2.2 Configure the app.....	6
2.2.3 Build & run	7
Stop / restart	7
2.2.4 Quick smoke tests (curl)	7
2.2.5 Browser/CORS (if testing from a static front-end)	8
2.2.6 Troubleshooting tips	8
Baseline MVC & Public Events API (Pre-JWT).....	9
3.1 Baseline MVC view is running.....	9
3.2 GET /api/events returns 200 OK.....	10
3.3 GET /api/events?q=... filters results.....	11
3.4 POST /api/events creates a new event	12
3.5 GET /api/events/{id} returns the created event	13
3.6 PUT /api/events/{id} updates the event.....	14
3.7 DELETE /api/events/{id} removes the event.....	15
3.8 GET /api/events/{id} after deletion returns 404	16
4. Users API (Pre-JWT).....	17
4.1 POST /api/users/register creates a new user.....	17
4.2 POST /api/users/login with valid credentials (returns user JSON pre-JWT)	18
4.3 POST /api/users/login with invalid credentials.....	19
5. JWT Configuration & Auth Flow (Post-JWT).....	20
5.1 JWT configuration added (secret + expiration)	20
5.2 Login returns JWT	21
5.3 Protected POST /api/events without token.....	22
5.4 Protected POST /api/events with token	23
5.5 Invalid JWT rejected	24
6. JWT-Protected Updates & Deletes.....	25
6.1 PUT /api/events/{id} without token.....	25

6.2 PUT /api/events/{id} with JWT	26
6.3 DELETE /api/events/{id} without token.....	27
6.4 DELETE /api/events/{id} with JWT.....	28
6.5 GET deleted /api/events/{id} (verification)	29
7. Organizer Ownership Enforcement (Claim → Model).....	30
7.1 POST with JWT auto-sets organizerid from uid claim.....	30
7.2 Server overrides spoofed organizerid.....	31
7.3 DB shows organizerid from JWT.....	32
8. CORS Verification	33
8.1 Preflight allows Authorization header	33
9. Duplicate Username Handling & DB Constraint	34
9.1 Duplicate username returns 409	34
9.2 Unique username index present	35
10. Summary of Key Concepts	36

1. Overview

This milestone extends the Events application with a clean, testable REST API and production-style security. I implemented two API surfaces—**Events** and **Users**—and layered **JWT authentication** so reads remain public while writes require a valid token. The site's original MVC pages continue to use form login, but the `/api/**` endpoints are now **stateless** and return proper **401/403** responses instead of HTML redirects. All changes were verified step-by-step in Postman and the terminal, with screenshots captured inline at the moment of each test.

At a glance:

- **REST endpoints:** Public GET `/api/events`, GET `/api/events/{id}`, and GET `/api/events?q=...`; protected POST/PUT/DELETE `/api/events/**`.
- **JWT flow:** POST `/api/users/login` issues a token; a `JwtAuthFilter` validates `Authorization: Bearer ...` and establishes the security context for API calls.
- **Ownership enforcement:** The server **derives `organizerid` from the JWT's `uid` claim**, ignoring any client-supplied value to prevent spoofing.
- **CORS:** Configured for `http://localhost:5500` and `http://127.0.0.1:5500`, including the `Authorization` header so a browser client can call protected endpoints.
- **Data integrity:** Registration rejects duplicate usernames with **409 Conflict**, and the database enforces a **UNIQUE** index on the username column.
- **Reliability fixes:** Null-safe model/entity converters, consistent 404 handling for missing resources, and method-specific security rules.

The section that follows presents environment setup, pre-JWT baseline tests, JWT integration and protected write operations, claim-to-model ownership, CORS verification, duplicate-user handling, and database constraints—each backed by the corresponding screenshot and exact request/response details.

2. Environment & Setup

2.1 Database configuration (datasource + JPA)

This section captures the exact configuration used to run the API locally. It includes the MySQL datasource (for both MVC and REST layers) and the JWT settings used by the stateless `/api/**` endpoints.

2.1.1 Database (MySQL)

Paste these into `src/main/resources/application.properties` (adjust host/port/user/pass if your setup differs):

```
# --- Datasource ---
spring.datasource.url=jdbc:mysql://127.0.0.1:8889/eventsapp?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=root
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# --- JPA (used for schema tooling / logging; persistence here is JdbcTemplate) ---
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

Notes

- `allowPublicKeyRetrieval=true` avoids auth negotiation issues on local MySQL.
- `serverTimezone=UTC` prevents timestamp warnings on some drivers.
- Even though the app uses `JdbcTemplate` repositories, keeping `ddl-auto=update` helps align the schema during development.

(If you included a screenshot of your DB settings, reference it here, e.g., “See Figure 2: Database configuration.”)

2.1.2 JWT (stateless API security)

These values power token creation/validation for `/api/**`. The same secret must be used to both sign and verify tokens.

```
# --- JWT ---
app.jwt.secret=PASTE_A_LONG_RANDOM_BASE64_STRING_ON_ONE_LINE
app.jwt.expirationMillis=3600000 # 1 hour
```

Important

- **Single line only.** Do not wrap the secret onto a second line (this will create a bogus property and break parsing).
- Generate a strong secret in Terminal, e.g.:
- `openssl rand -base64 64`
- `expirationMillis` controls token lifetime; $3,600,000 = 1$ hour (Activity 6 parity).

(Reference your JWT screenshot here, e.g., “See Figure 11: JWT configuration added.”)

2.1.3 Quick validation checklist

- App starts: run `mvn spring-boot:run` and confirm “Started ... on port 8080”.
- DB reachable: no datasource exceptions in the startup log.
- JWT key length: no “weak key”/“key too short” errors on login.
- Secret is a **single** line in `application.properties` (no accidental newline between base64 chunks).

2.2 How to run (port 8080, MySQL schema)

This is exactly how I ran the app locally to produce the screenshots in later sections.

2.2.1 Prerequisites

- **Java** 17 or 21 (LTS)
- **Maven** 3.8+ (`mvn -v`)
- **MySQL** running with a schema named `eventsapp`
 - User/password must match `application.properties` (default in this project is `root / root`)
 - Host/port default to `127.0.0.1:8889` (update if your MySQL port is different)

If the `eventsapp` database doesn’t exist yet, create it:

```
CREATE DATABASE IF NOT EXISTS eventsapp CHARACTER SET utf8mb4 COLLATE
utf8mb4_0900_ai_ci;
```

2.2.2 Configure the app

Confirm `src/main/resources/application.properties` matches your environment (see 2.1).
Key settings:

- `spring.datasource.url=jdbc:mysql://127.0.0.1:8889/eventsapp?...`
- `spring.datasource.username=root`
- `spring.datasource.password=root`

- `app.jwt.secret=<one-line random base64>`
- `app.jwt.expirationMillis=3600000`

2.2.3 Build & run

From the project root:

```
mvn clean spring-boot:run
```

Expected console line (toward the end):

```
Started EventsappApplication in ... seconds (JVM running for ...)
```

The server listens on <http://localhost:8080>.

Stop / restart

- Stop: press `Ctrl+C` in the terminal
- Rebuild only: `mvn -q -DskipTests package`
- Run again: `mvn spring-boot:run`

2.2.4 Quick smoke tests (curl)

These confirm the API is reachable before opening Postman. Replace `{id}` as indicated.

1. Public GET (no token)

```
curl -i http://localhost:8080/api/events
```

Expect `HTTP/1.1 200` and a JSON array.

2. Register a user (only once per username)

```
curl -i -X POST http://localhost:8080/api/users/register \
-H "Content-Type: application/json" \
-d '{"userName":"alex","password":"password123"}'
```

Expect `HTTP/1.1 201`.

3. Login → get JWT token

```
curl -s -X POST http://localhost:8080/api/users/login \
-H "Content-Type: application/json" \
-d '{"userName":"alex","password":"password123"}'
```

Expect `{"token":"<eyJhbGciOi...>"}`, copy the token value (without quotes).

4. Protected POST (with token)

```
TOKEN=<paste_token_here>
curl -i -X POST http://localhost:8080/api/events \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"name":"Family Camp","description":"JWT create test","location":"Mt. Lemmon, AZ","date":"2025-09-15"}'
```

Expect HTTP/1.1 201 and a JSON body.

Note: organizerid is **auto-set from the JWT**; you do not send it in the body.

5. Public GET by id (no token)

```
curl -i http://localhost:8080/api/events/{id}
```

Expect HTTP/1.1 200 for an existing id, 404 if not found.

2.2.5 Browser/CORS (if testing from a static front-end)

If you call the API from a page served at `http://localhost:5500` (or `127.0.0.1:5500`), the CORS configuration in this project already allows:

- **Methods:** GET, POST, PUT, DELETE, OPTIONS
- **Headers:** Authorization, Content-Type

You must include the **Authorization** header in XHR/fetch calls:

```
Authorization: Bearer <token>
```

2.2.6 Troubleshooting tips

- **Communications link failure OR Access denied:** verify MySQL is running, host/port/user/pass match `application.properties`, and the `eventsapp` schema exists.
- **JWT login returns HTML instead of 401/403:** make sure the split security chains are in place (API vs MVC). The API chain must be stateless and use `HttpStatusEntryPoint(HttpStatus.UNAUTHORIZED)`.
- **key is too short on JWT:** regenerate a secret that's at least 64 base64 chars and keep it on **one line**.
- **Incorrect integer value: 'null' for column 'organizerid':** call protected endpoints **with a token**; the server sets `organizerid` from the JWT uid.

That's it, once these checks pass, you can reproduce every figure in the later sections using either Postman or the curl commands above.

3.1 Baseline MVC view is running

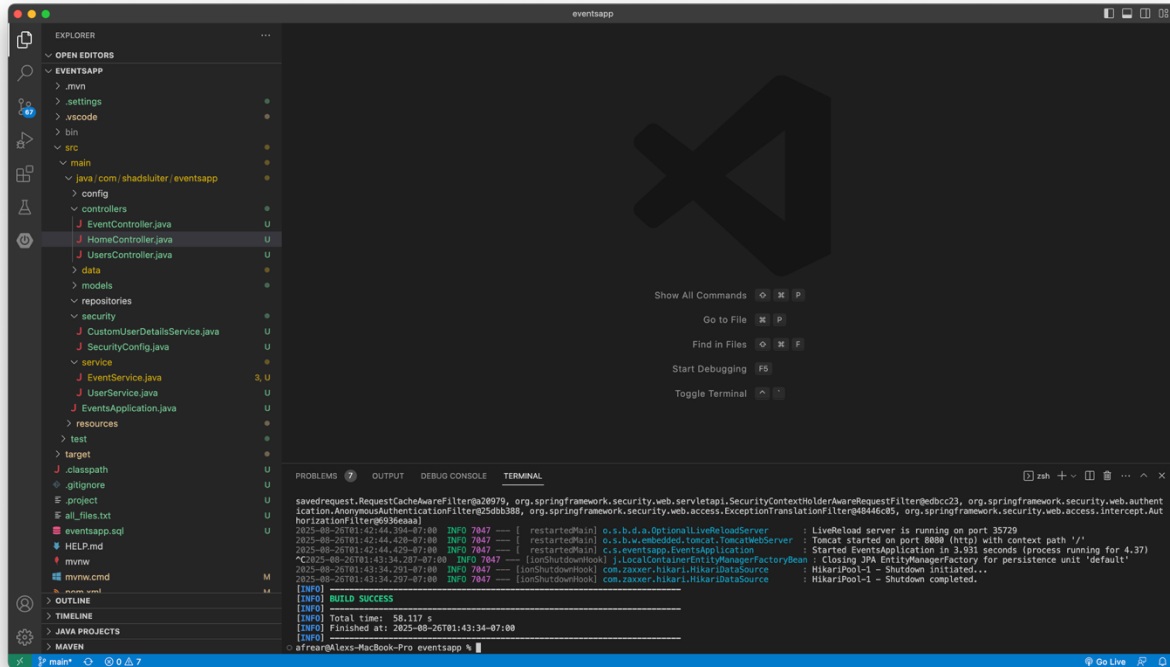


Figure 1 The pre-existing MVC events list renders successfully before adding REST endpoints.

3.2 GET /api/events returns 200 OK

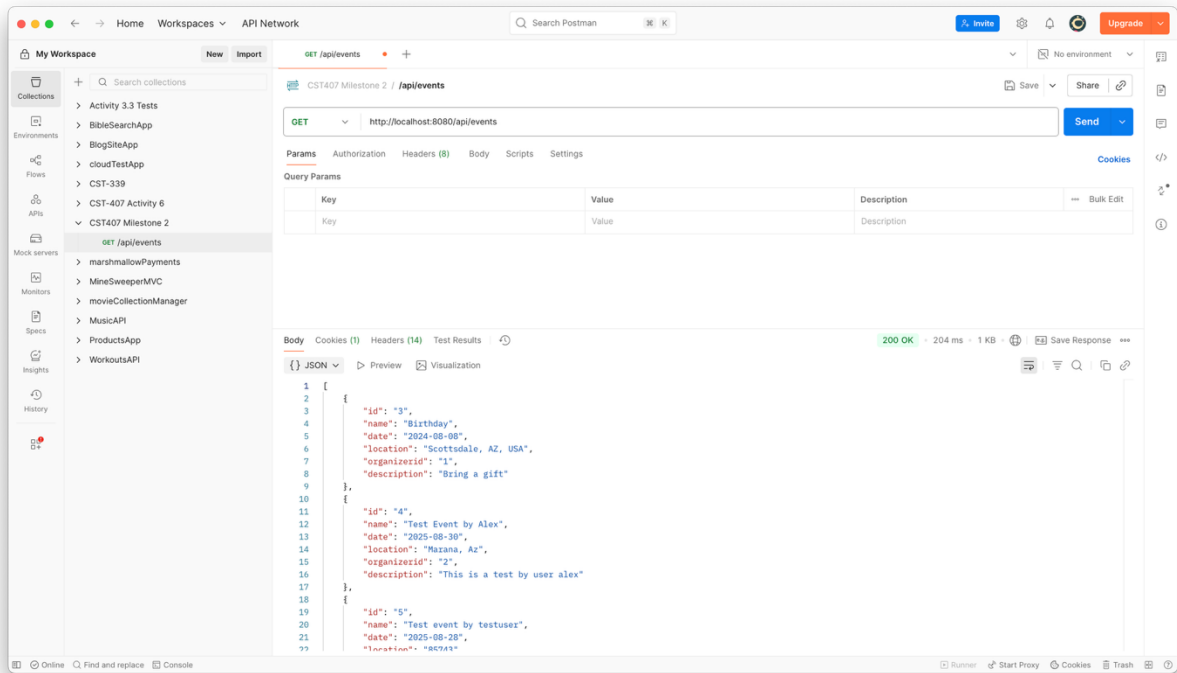


Figure 2 Postman GET to /api/events shows a 200 OK response with the JSON array of events.

3.3 GET /api/events?q=... filters results

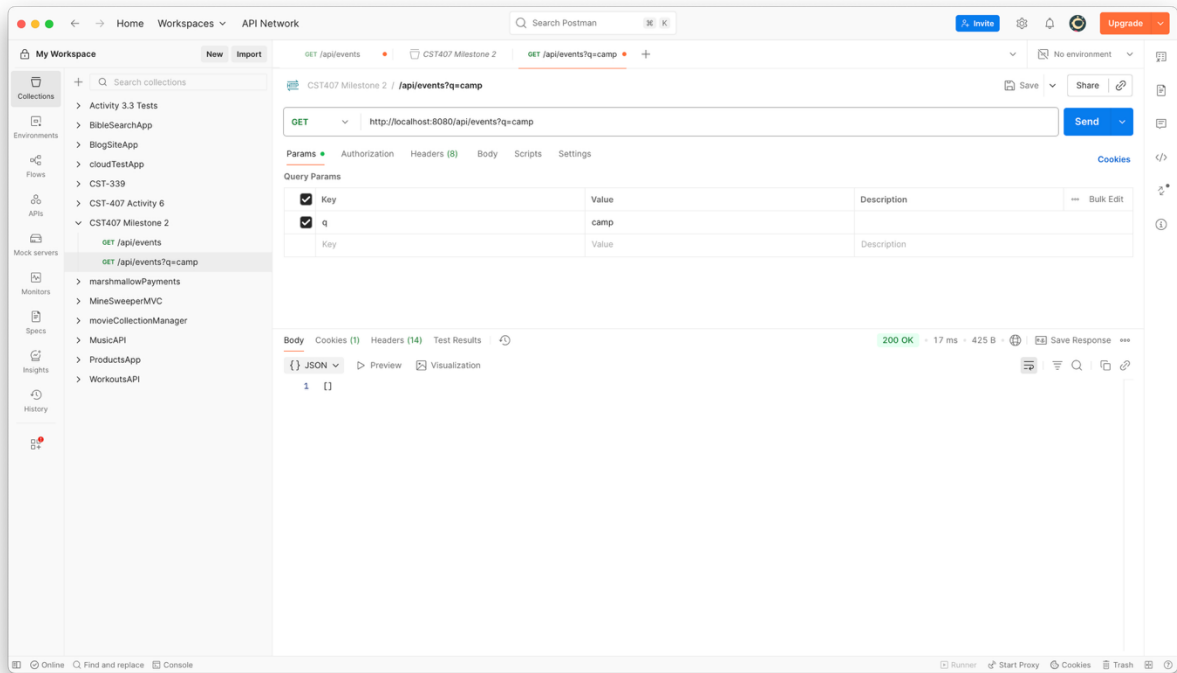


Figure 3 Postman GET to /api/events with query parameter q filters results across name, description, or location.

3.4 POST /api/events creates a new event

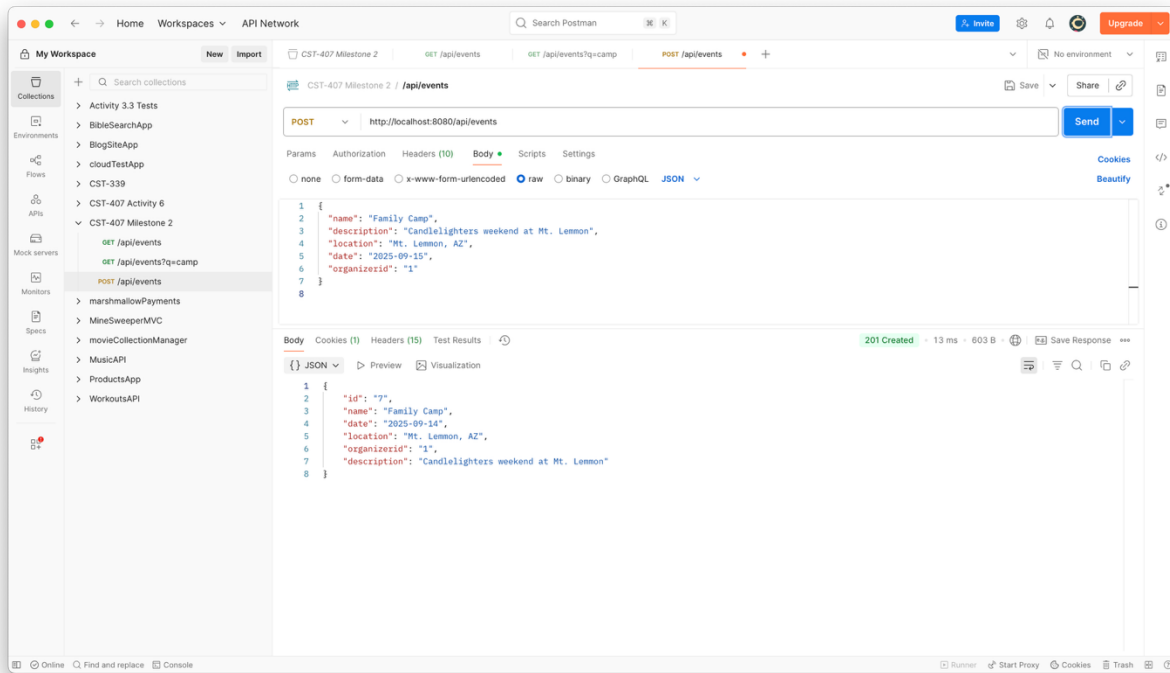


Figure 4 Postman POST to /api/events returns 201 Created with a Location header and the created event payload.

3.5 GET /api/events/{id} returns the created event

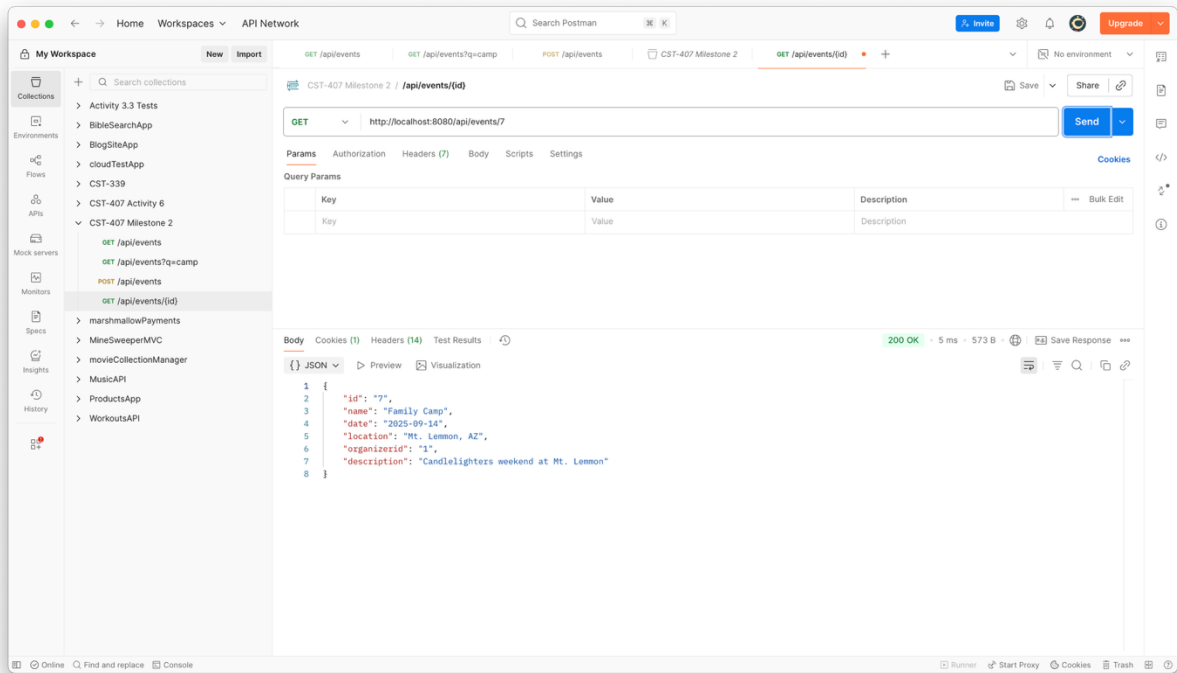


Figure 5 Postman GET to /api/events/{id} returns 200 OK and the JSON for the created event.

3.6 PUT /api/events/{id} updates the event

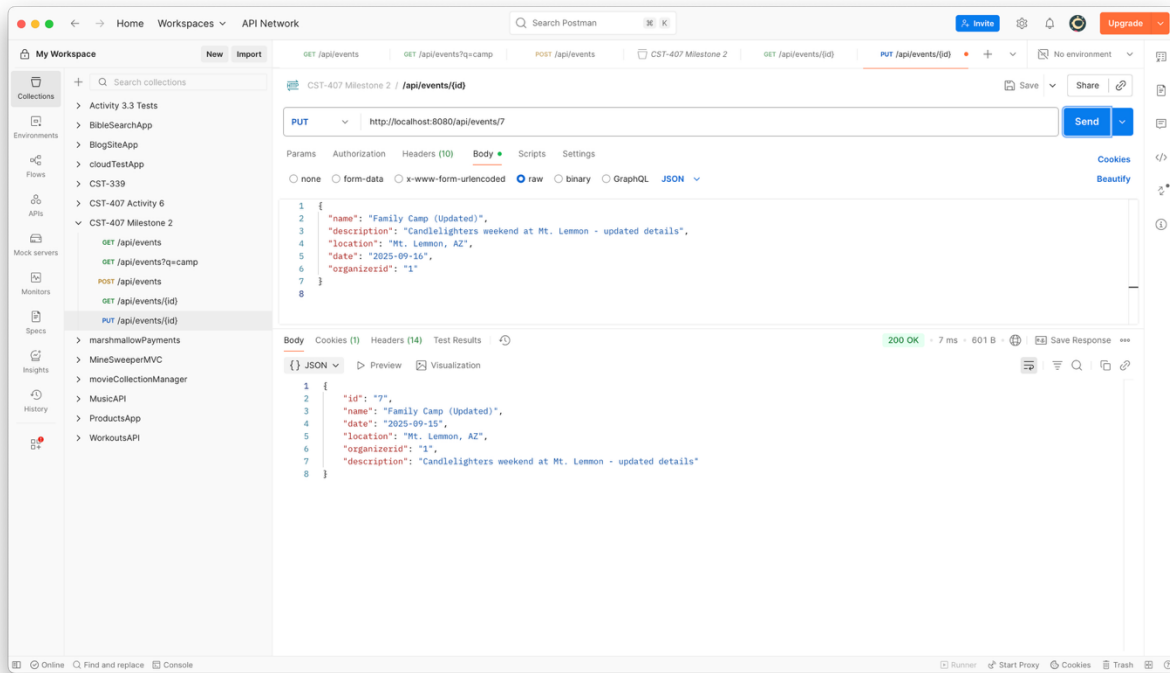


Figure 6 Postman PUT to /api/events/{id} returns 200 OK with the updated event payload.

3.7 DELETE /api/events/{id} removes the event

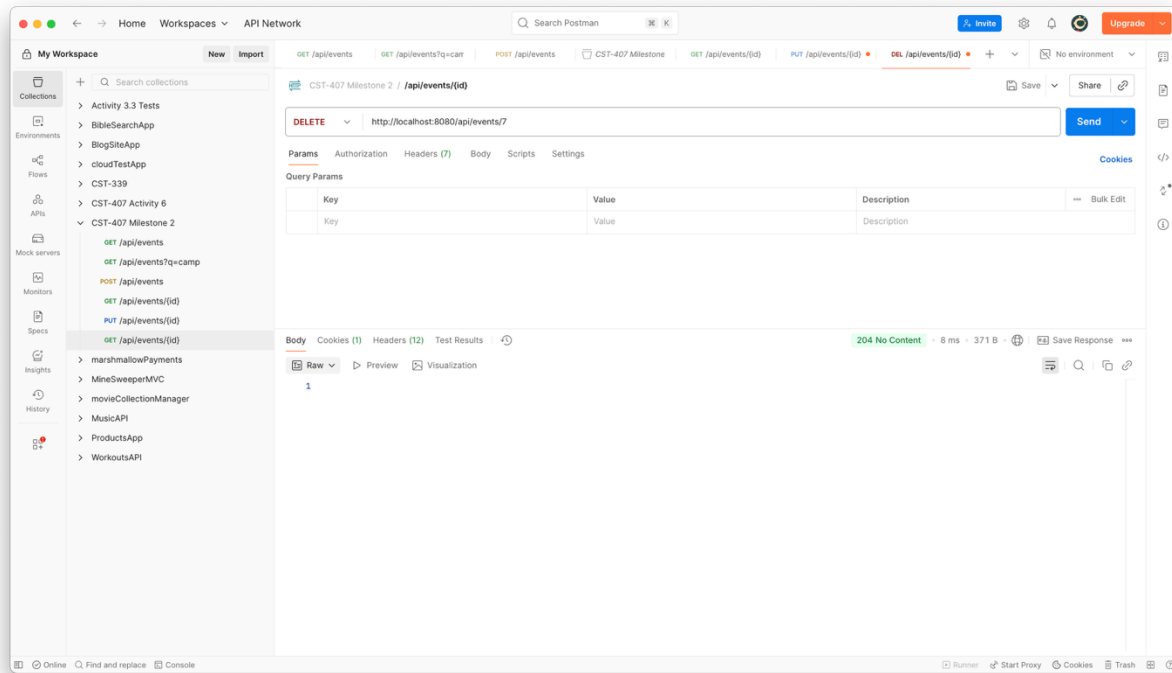


Figure 7 Postman DELETE to /api/events/{id} returns 204 No Content.

3.8 GET /api/events/{id} after deletion returns 404

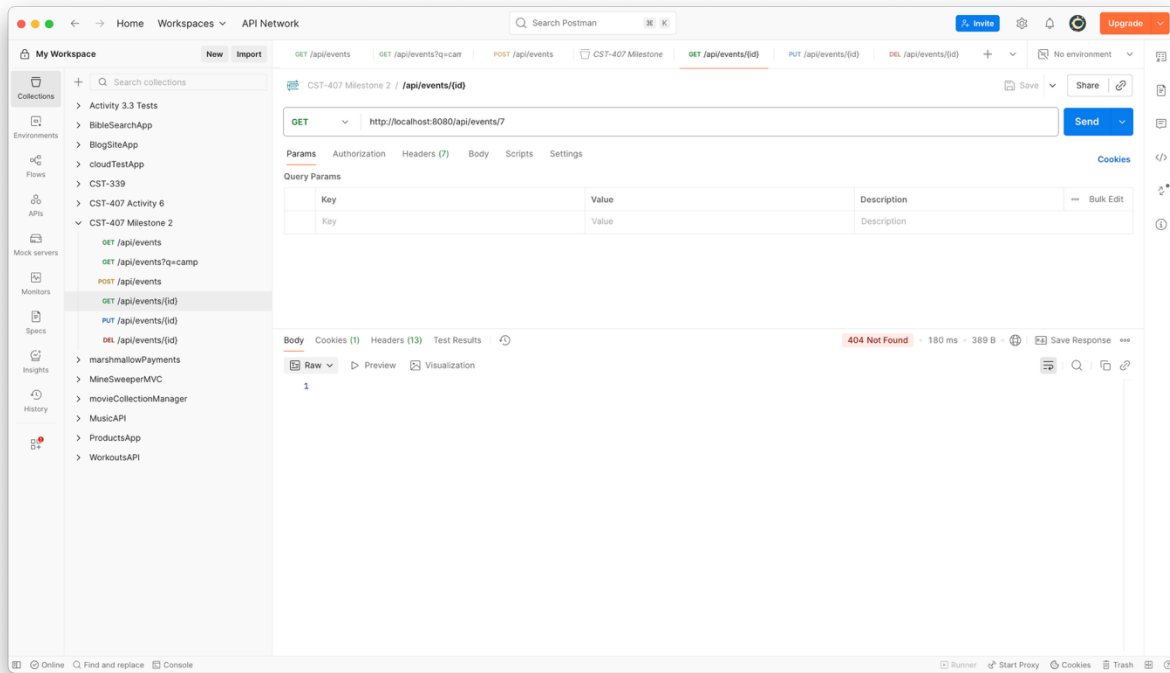


Figure 8 Postman GET to /api/events/{id} after deletion returns 404 Not Found.

4. Users API (Pre-JWT)

4.1 POST /api/users/register creates a new user

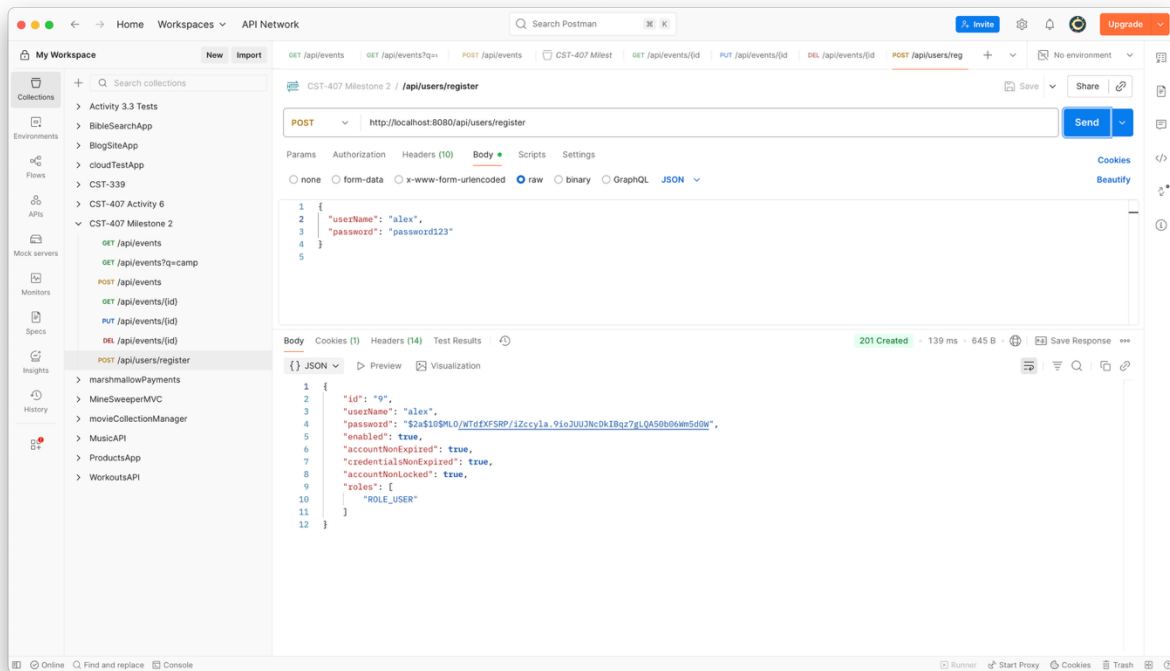


Figure 9 Postman POST to /api/users/register with username and password returns 201 Created and the created user payload.

4.2 POST /api/users/login with valid credentials (returns user JSON pre-JWT)

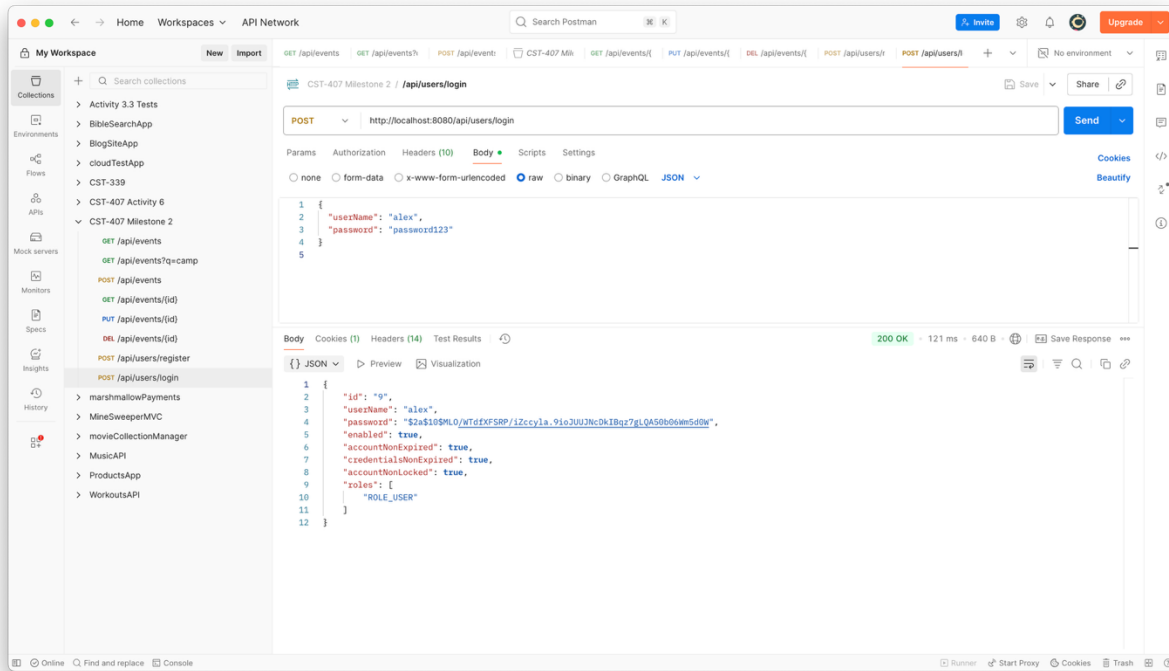


Figure 10 Postman POST to /api/users/login with valid credentials returns 200 OK and the user JSON.

4.3 POST /api/users/login with invalid credentials

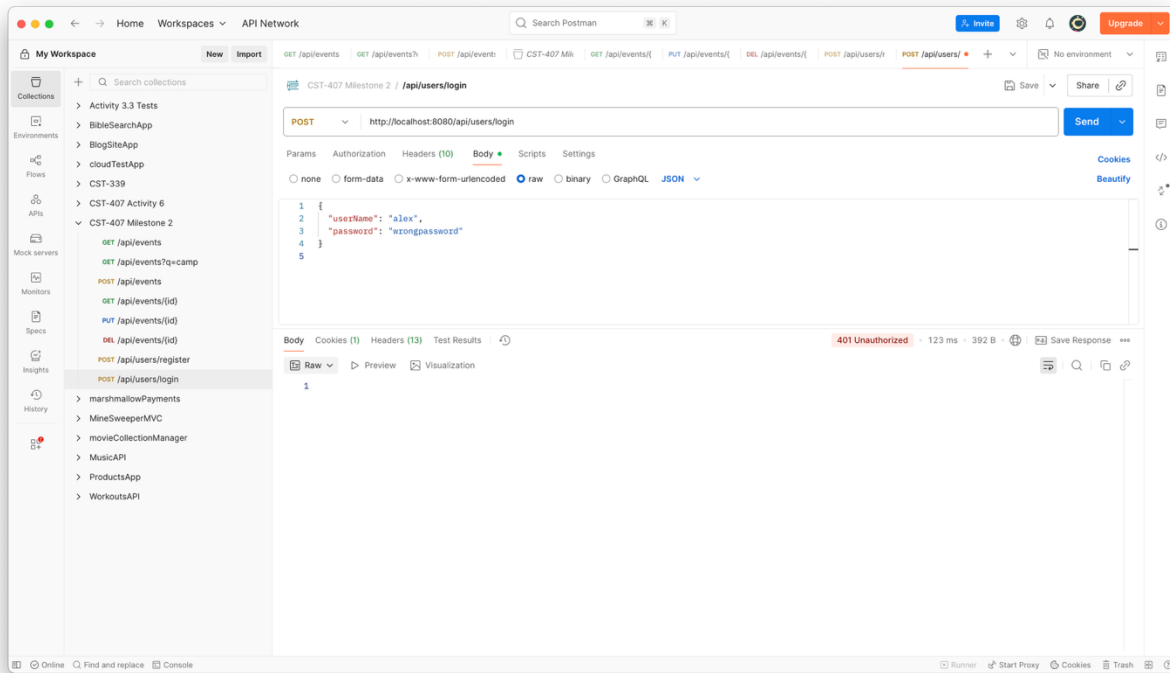


Figure 11 Postman POST to /api/users/login with incorrect password returns 401 Unauthorized.

5.1 JWT configuration added (secret + expiration)



5.2 Login returns JWT

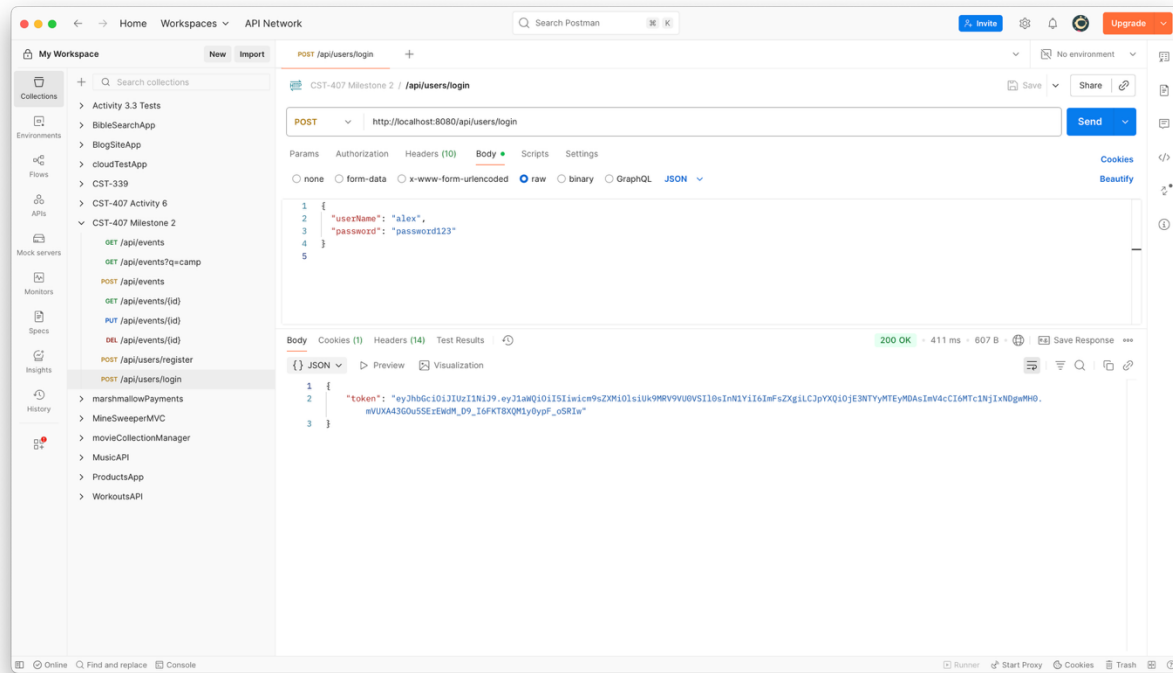


Figure 13 POST /api/users/login returns 200 OK with a JWT token in the response body.

5.3 Protected POST /api/events without token

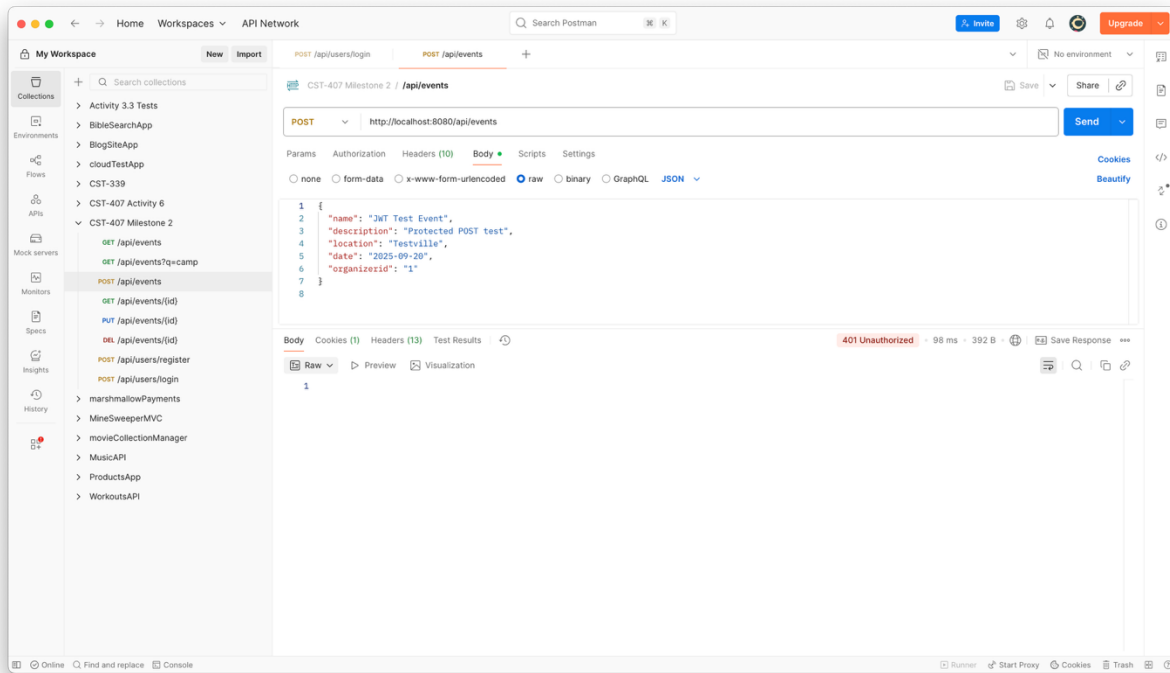


Figure 14 POST /api/events without Authorization header returns 401 Unauthorized.

5.4 Protected POST /api/events with token

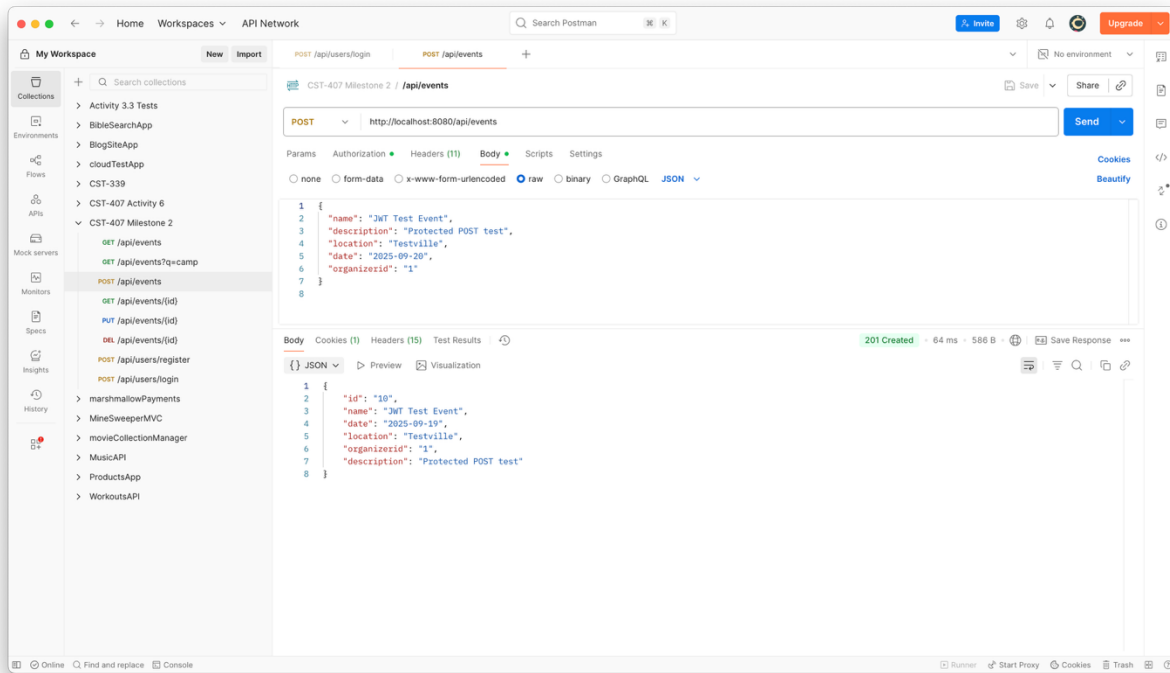


Figure 15 POST /api/events with Authorization: Bearer <token> returns 201 Created.

5.5 Invalid JWT rejected

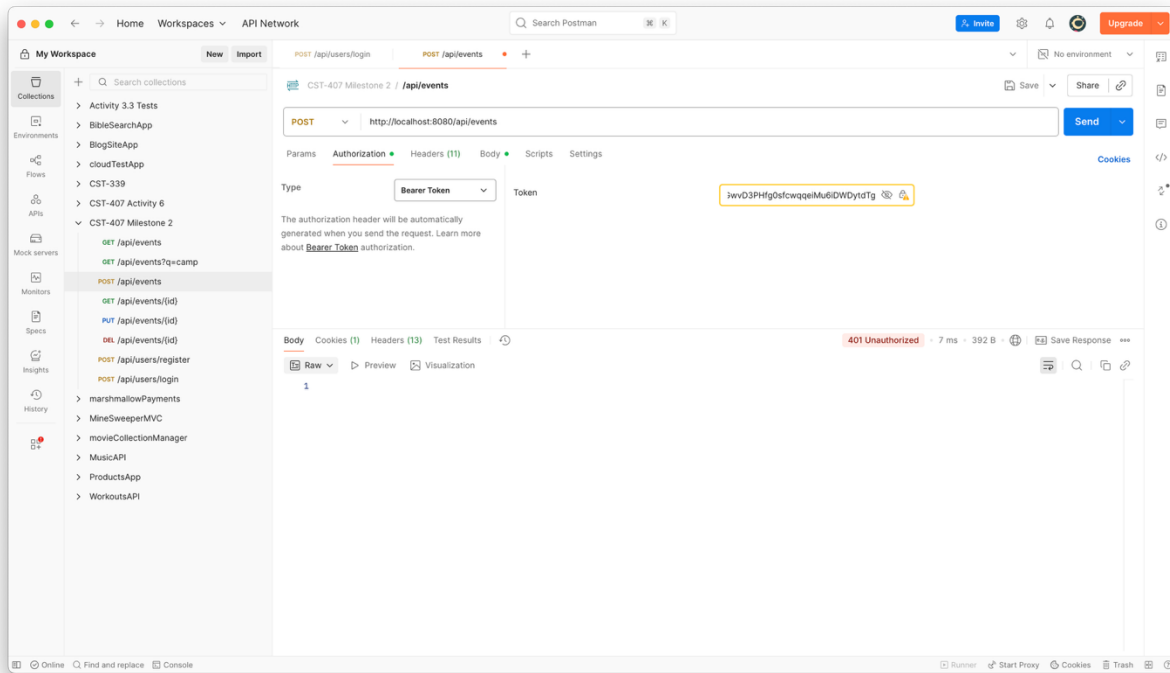


Figure 16 POST /api/events with an invalid JWT returns 401 Unauthorized.

6. JWT-Protected Updates & Deletes

6.1 PUT /api/events/{id} without token

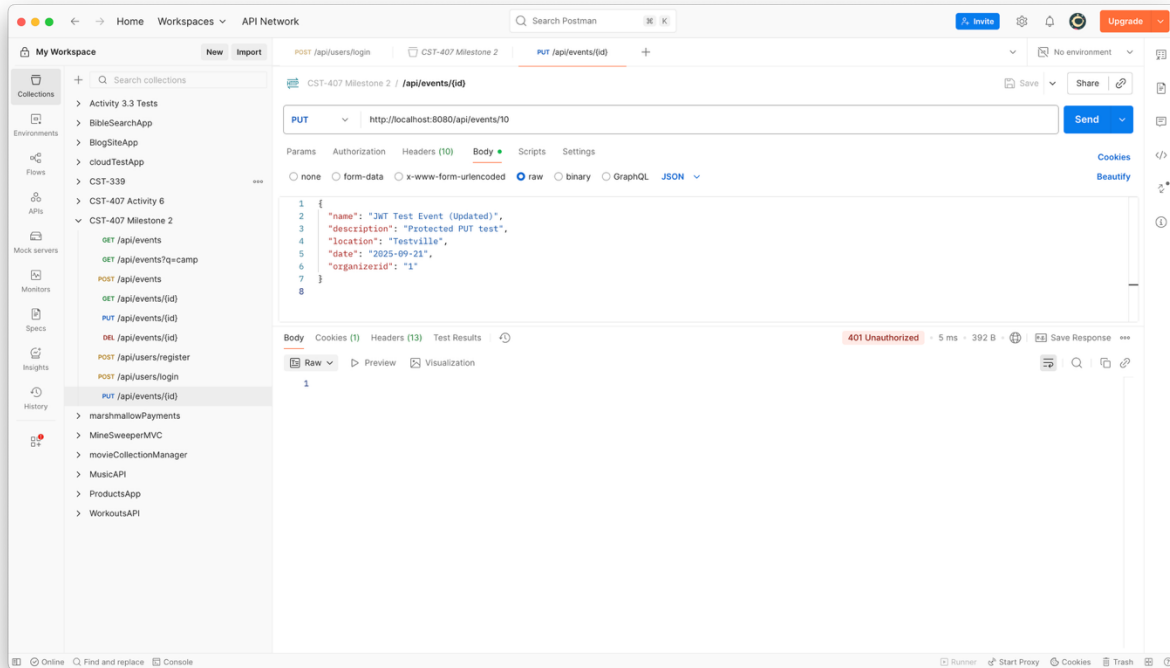


Figure 17 PUT /api/events/{id} without Authorization header returns 401 Unauthorized.

6.2 PUT /api/events/{id} with JWT

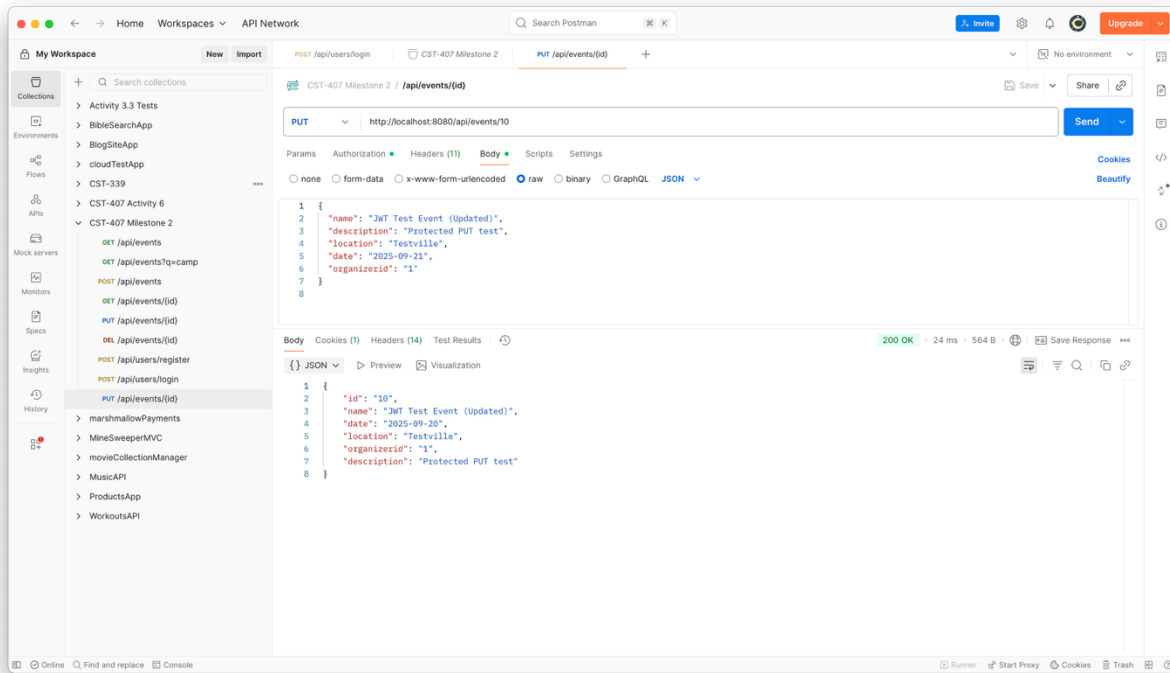


Figure 18 PUT /api/events/{id} with Authorization: Bearer <token> returns 200 OK and the updated event payload.

6.3 DELETE /api/events/{id} without token

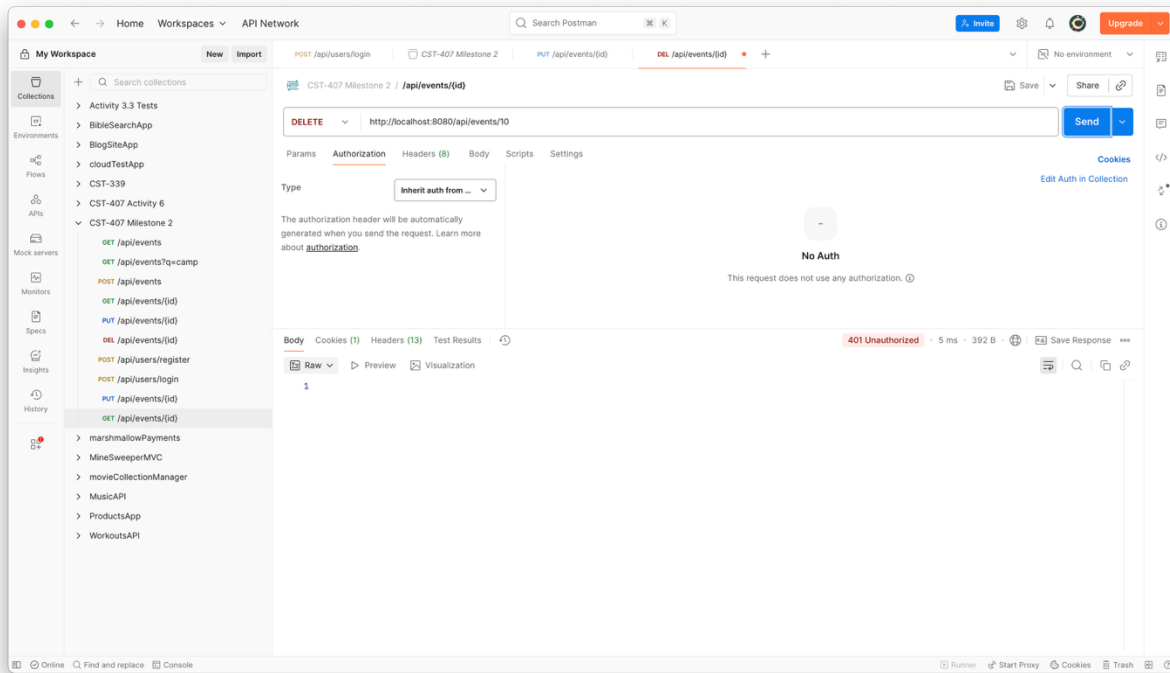


Figure 19 DELETE /api/events/{id} without Authorization header returns 401 Unauthorized.

6.4 DELETE /api/events/{id} with JWT

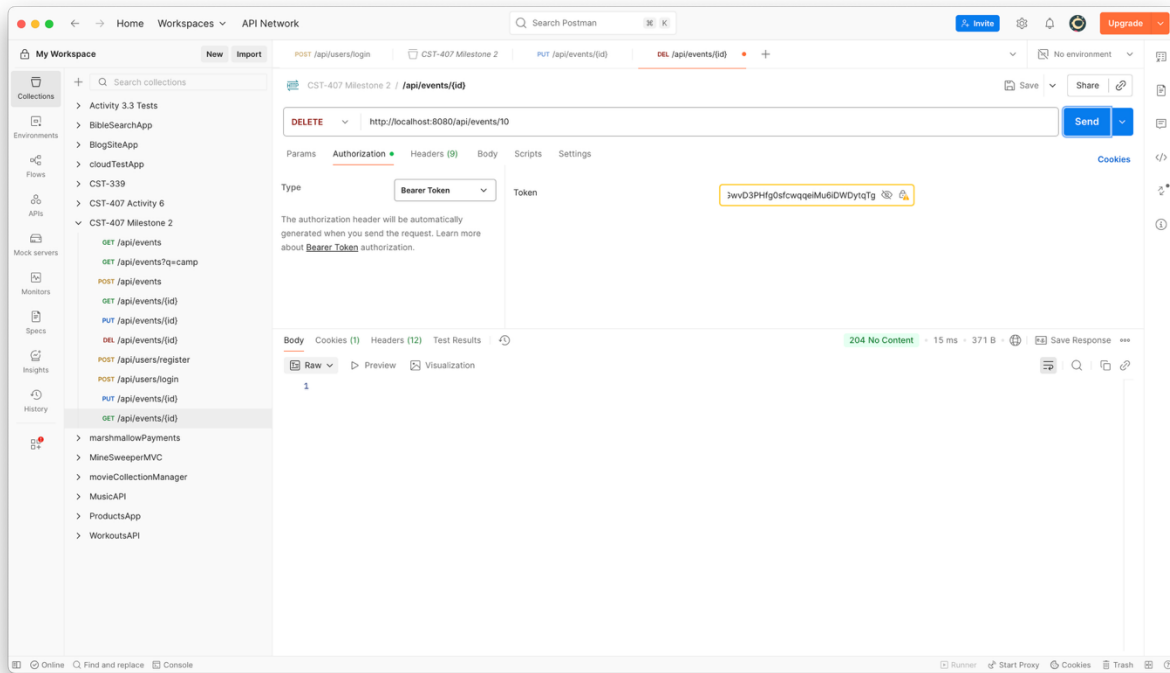


Figure 20 DELETE /api/events/{id} with Authorization: Bearer <token> returns 204 No Content.

6.5 GET deleted /api/events/{id} (verification)

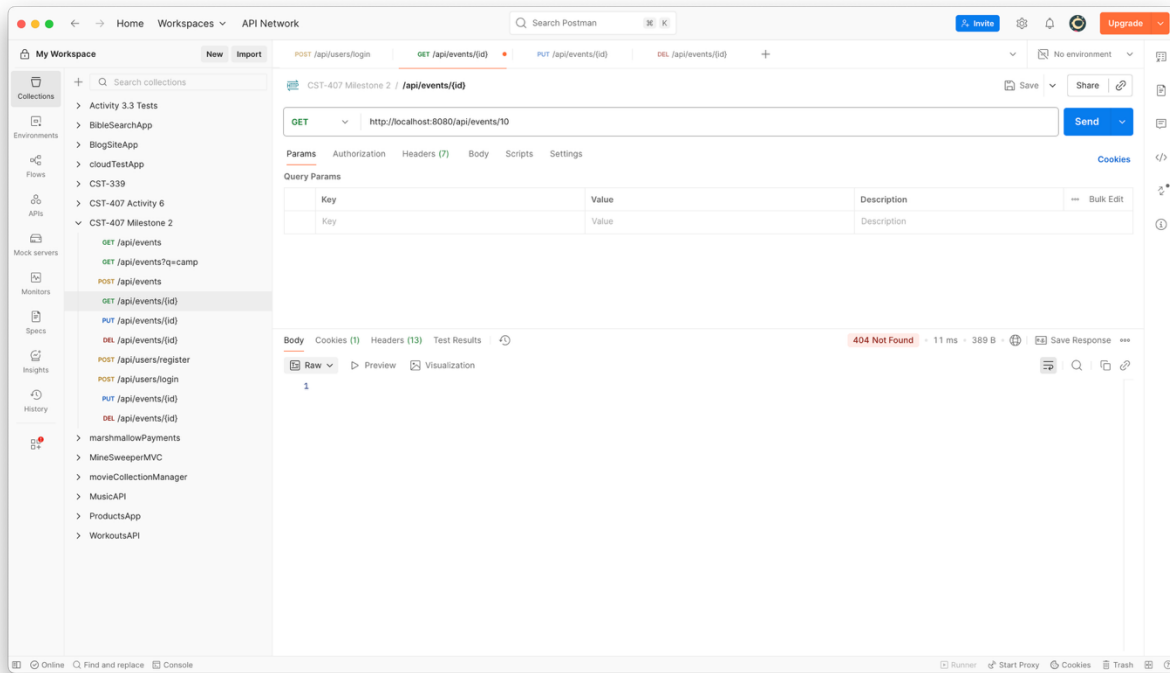


Figure 21 Public GET after deletion returns 404 Not Found, confirming the resource was removed.

7. Organizer Ownership Enforcement (Claim → Model)

7.1 POST with JWT auto-sets organizerid from uid claim

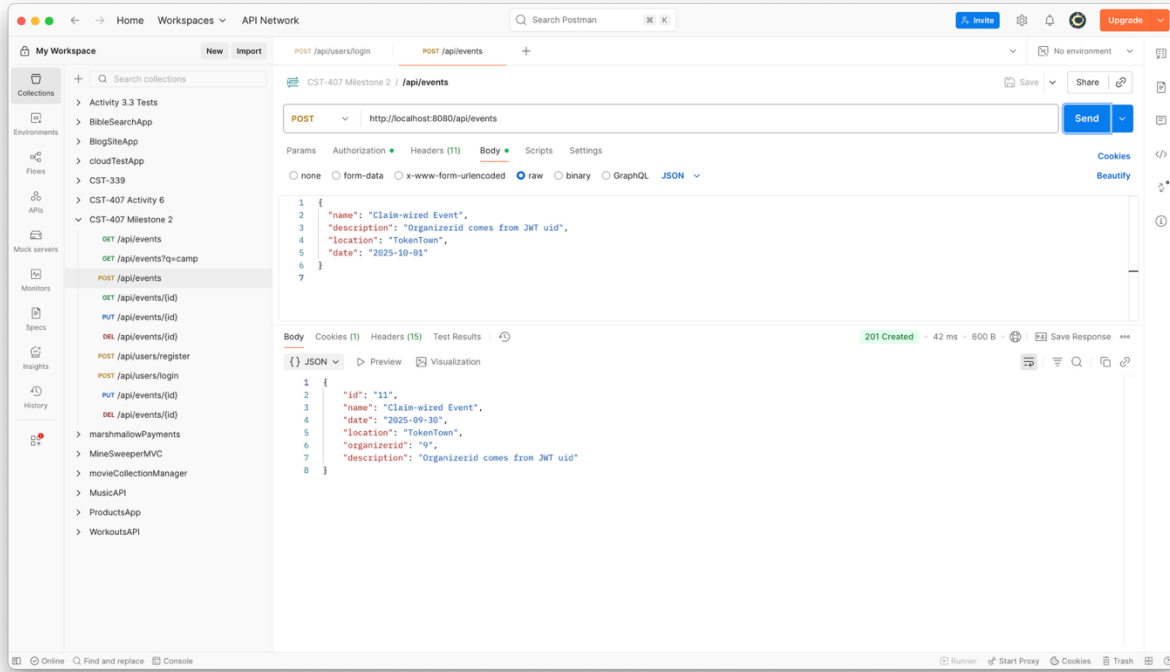


Figure 22 POST /api/events with a JWT (no organizerid in body) returns 201 Created, and the server populates organizerid from the token's uid claim.

7.2 Server overrides spoofed organizerid

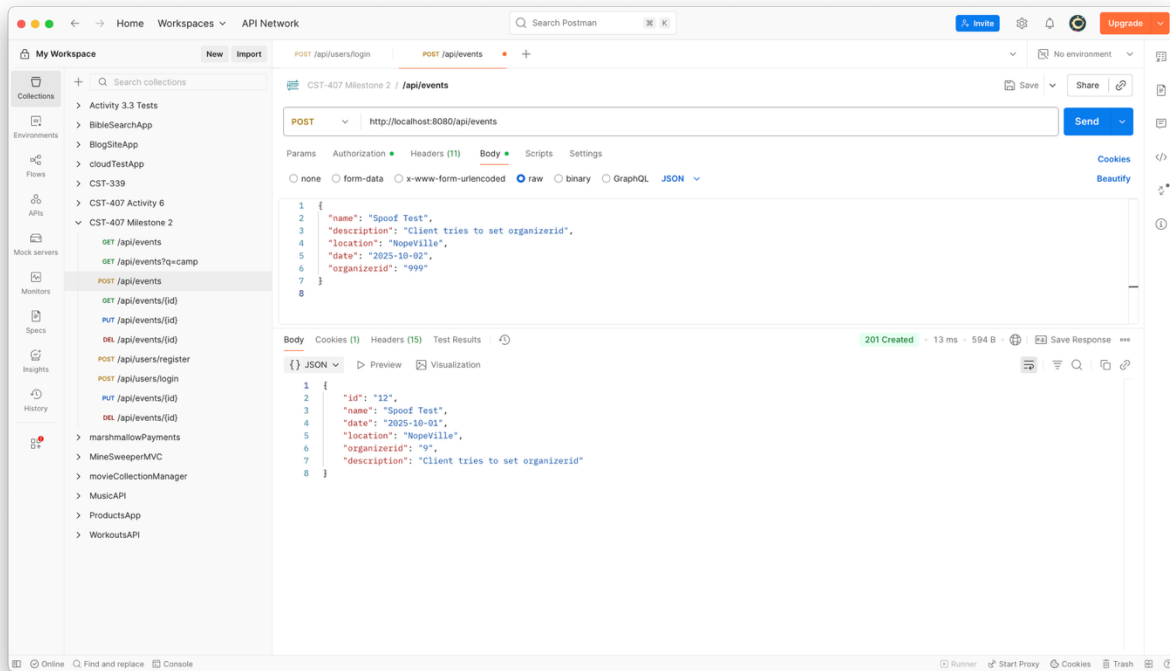


Figure 23 Even when the client sends `organizerid=999`, the server overwrites it with the `uid` from the JWT claim.

7.3 DB shows organizerid from JWT

The screenshot displays the MySQL Workbench interface. The top toolbar shows various database management icons. The left sidebar contains the 'SCHEMAS' panel with a tree view showing the database structure: 'eventsapp' (Tables: events, roles, users; Views; Stored Procedures; Functions), 'MedicalJourneyLog', 'ordersappjwt', 'ordersapproles', and 'sys'. The main query editor shows a SQL query: `SELECT id, name, organizerid FROM events ORDER BY id DESC LIMIT 5;`. The 'Result Grid' below the query shows the results of the query, with columns 'id', 'name', and 'organizerid'. The results are as follows:

id	name	organizerid
12	Spod Test	9
11	Claim-wired Event	9
6	Test event by admin	3
3	Birthday	1

The bottom panel shows the 'Action Output' tab, which displays a list of database actions and their responses. The actions include:

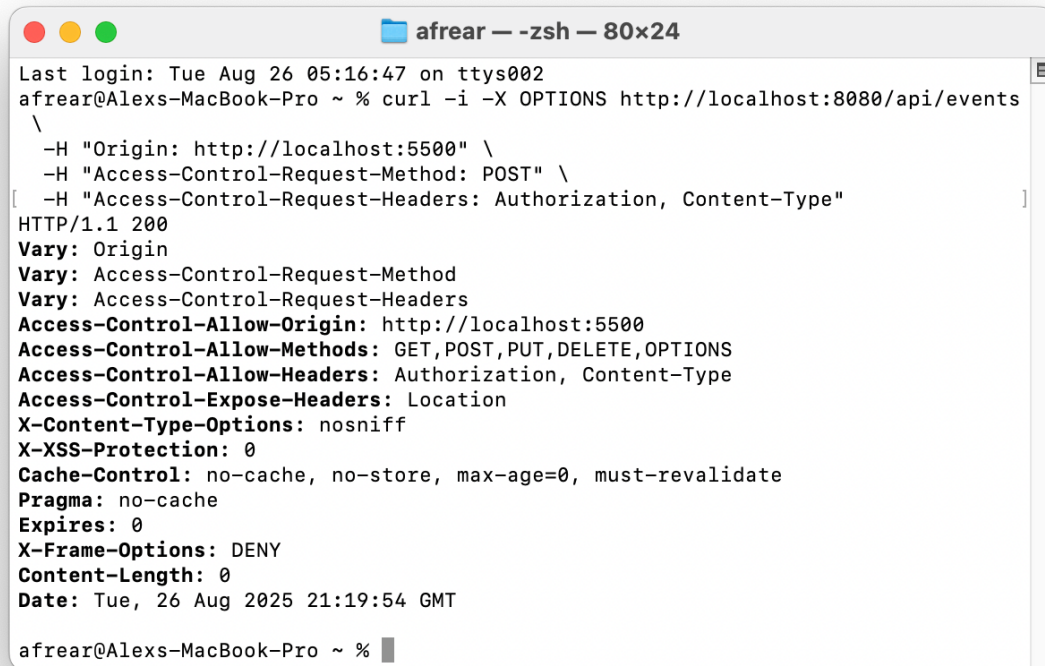
- 39 03:47:25 DELETE FROM roles WHERE user_id = 4
- 40 03:47:25 DELETE FROM users WHERE id = 4
- 41 03:47:29 Refresh Recordset
- 42 03:47:34 DELETE FROM 'eventsapp'.'users' WHERE ('id' = '4')
- 43 03:47:43 SELECT * FROM eventsapp.users LIMIT 0, 1000
- 44 03:47:53 SELECT * FROM eventsapp.roles LIMIT 0, 1000
- 45 03:48:03 SELECT * FROM eventsapp.events LIMIT 0, 1000
- 46 03:48:25 DELETE FROM 'eventsapp'.'users' WHERE ('id' = '4')
- 47 14:13:16 SELECT id, name, organizerid FROM events ORDER BY id DESC LIMIT 5

The 'Response' column shows the results of these actions, including the number of rows affected, error messages, and the duration of the actions.

Figure 24 SELECT query shows organizerid values populated from the JWT uid claim for newly created rows.

8. CORS Verification

8.1 Preflight allows Authorization header



```
afrear — -zsh — 80x24
Last login: Tue Aug 26 05:16:47 on ttys002
afrear@Alexs-MacBook-Pro ~ % curl -i -X OPTIONS http://localhost:8080/api/events \
  -H "Origin: http://localhost:5500" \
  -H "Access-Control-Request-Method: POST" \
  -H "Access-Control-Request-Headers: Authorization, Content-Type"
HTTP/1.1 200
Vary: Origin
Vary: Access-Control-Request-Method
Vary: Access-Control-Request-Headers
Access-Control-Allow-Origin: http://localhost:5500
Access-Control-Allow-Methods: GET,POST,PUT,DELETE,OPTIONS
Access-Control-Allow-Headers: Authorization, Content-Type
Access-Control-Expose-Headers: Location
X-Content-Type-Options: nosniff
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
X-Frame-Options: DENY
Content-Length: 0
Date: Tue, 26 Aug 2025 21:19:54 GMT
afrear@Alexs-MacBook-Pro ~ %
```

Figure 25 `OPTIONS` preflight from Origin `http://localhost:5500` returns the expected `Access-Control-Allow-*` headers including `Authorization`.

9. Duplicate Username Handling & DB Constraint

9.1 Duplicate username returns 409

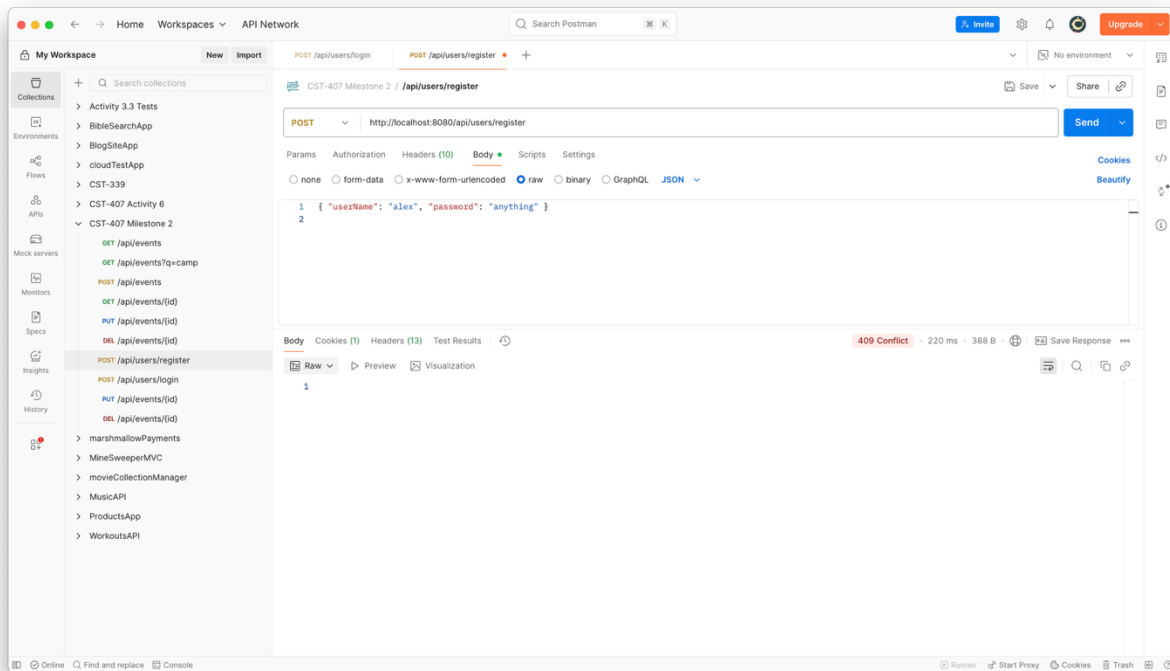
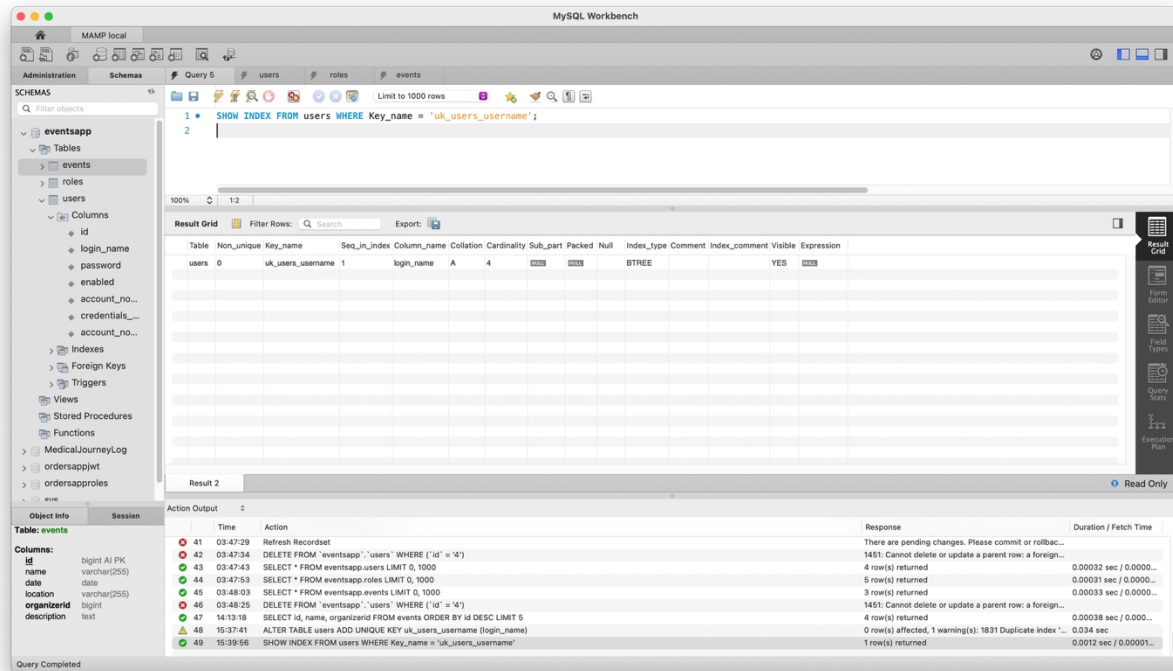


Figure 26 Attempting to register an existing username returns 409 Conflict.

9.2 Unique username index present



The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'schemas' tree with 'eventsapp' selected. The 'users' table is highlighted under 'Tables'. The 'Columns' list for 'users' includes 'id', 'login_name', 'password', 'enabled', 'account_no...', 'credentials...', and 'account_no...'. The 'Indexes' section shows a unique index 'uk_users_username' on the 'login_name' column. The 'Query' tab is active, showing the query: `SHOW INDEX FROM users WHERE Key_name = 'uk_users_username';`. The 'Result Grid' displays the query results, showing a single row for the index 'uk_users_username' on the 'login_name' column, with 'Non_unique' set to 0, indicating a unique index. The 'Action Output' tab at the bottom shows the execution log, including the query execution and the response: '1 row(s) returned'.

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible	Expression
users	0	uk_users_username	1	login_name	A	4				BTREE			YES	

Figure 27 `SHOW INDEX` confirms `uk_users_username` exists on the `users.login_name` column with `Non_unique = 0` (unique).

10. Summary of Key Concepts

I came into this milestone with a little experience from previous classes—mostly basic CRUD with controllers and services—so the biggest leap for me was adding real authentication and authorization with JWT while keeping the site’s MVC pages working. The clean separation helped: public GET endpoints stayed open, while POST/PUT/DELETE under `/api/events/**` now require a token. Once I saw the pattern in Postman—401 without `Authorization: Bearer <token>` and success with it—the security model clicked.

Another key idea was splitting security into two chains. I’d used form login before, but I hadn’t configured a separate, stateless API chain. Making `/api/**` return proper 401/403 (instead of HTML redirects) and keeping form login just for the MVC pages solved a lot of confusion. Plugging in the `JwtAuthFilter` taught me how the token ties into `SecurityContext`, and why API security shouldn’t rely on session state.

I also learned to derive ownership from claims instead of trusting client input. The server reads the user id (`uid`) from the token and sets `organizerid` server-side, which prevents spoofing and simplifies clients (they don’t send that field). On the browser side, enabling CORS for my dev origin and allowing the `Authorization` header was straightforward but essential for making authenticated requests from a front-end.

Finally, I strengthened data integrity and error handling. Registration returns 409 Conflict on duplicate usernames, backed by a `UNIQUE` index in the database. Converters are now null-safe (no crashes on inserts with missing ids), and the repository “not found” cases surface as 404 instead of 500. The biggest process lesson was to move in small steps—implement, test in Postman or curl, screenshot, then proceed. That cadence kept the work organized and made the final results easy to verify.